Embedded Operating System HW04 <A111193>

Screenshot of your source code

1. The original USB HID driver with all your comments.

```c
// SPDX-License-Identifier: GPL-2.0-or-later
/*
 *  Copyright (c) 1999-2001 Vojtech Pavlik
 *
 *  USB HIDBP Mouse support
 */

/*
 *
 * Should you need to contact me, the author, you can do so either by
 * e-mail - mail your message to <vojtech@ucw.cz>, or by paper mail:
 * Vojtech Pavlik, Simunkova 1594, Prague 8, 182 00 Czech Republic
 */

#include <linux/kernel.h>
#include <linux/slab.h>
#include <linux/module.h>
#include <linux/init.h>
#include <linux/usb/input.h>
#include <linux/hid.h>

// Conditional inclusion for Apple device IDs
#ifdef CONFIG_USB_HID_MODULE
#include "../hid-ids.h"
#endif

/*
 * Version Information
 */
#define DRIVER_VERSION "v1.6"
#define DRIVER_AUTHOR "Vojtech Pavlik <vojtech@ucw.cz>"
#define DRIVER_DESC "USB HID Boot Protocol mouse driver"

MODULE_AUTHOR(DRIVER_AUTHOR);        // Sets the author of the module
MODULE_DESCRIPTION(DRIVER_DESC);     // Sets the description of the module
MODULE_LICENSE("GPL");               // Sets the license of the module to GPL

// Structure representing the USB mouse
struct usb_mouse {
    char name[128];                   // Name of the device
    char phys[64];                    // Physical path of the device
    struct usb_device *usbdev;        // Pointer to the USB device
    struct input_dev *dev;            // Pointer to the input device
    struct urb *irq;                  // USB request block for interrupt transfer

    signed char *data;                // Pointer to data buffer
    dma_addr_t data_dma;              // DMA address of the data buffer
};

// Interrupt handler for USB mouse
static void usb_mouse_irq(struct urb *urb)
{
    struct usb_mouse *mouse = urb->context;
    signed char *data = mouse->data;
    struct input_dev *dev = mouse->dev;
    int status;

    switch (urb->status) {
    case 0:         /* success */
        break;
    case -ECONNRESET: /* unlink */
    case -ENOENT:
    case -ESHUTDOWN:
        return;
    /* -EPIPE:  should clear the halt */
    default:        /* error */
        goto resubmit;
    }

    // Report button and movement events
    input_report_key(dev, BTN_LEFT,   data[0] & 0x01);
    input_report_key(dev, BTN_RIGHT,  data[0] & 0x02);
    input_report_key(dev, BTN_MIDDLE, data[0] & 0x04);
    input_report_key(dev, BTN_SIDE,   data[0] & 0x08);
    input_report_key(dev, BTN_EXTRA,  data[0] & 0x10);

    input_report_rel(dev, REL_X,     data[1]);
    input_report_rel(dev, REL_Y,     data[2]);
    input_report_rel(dev, REL_WHEEL, data[3]);

    input_sync(dev);  // Synchronize input events
resubmit:
    status = usb_submit_urb (urb, GFP_ATOMIC);
    if (status)
        dev_err(&mouse->usbdev->dev,
            "can't resubmit intr, %s-%s/input0, status %d\n",
            mouse->usbdev->bus->bus_name,
            mouse->usbdev->devpath, status);
}

// Open function for the input device
static int usb_mouse_open(struct input_dev *dev)
{
    struct usb_mouse *mouse = input_get_drvdata(dev);

    mouse->irq->dev = mouse->usbdev;
    if (usb_submit_urb(mouse->irq, GFP_KERNEL))
        return -EIO;

    return 0;
}

```

```
// Close function for the input device
static void usb_mouse_close(struct input_dev *dev)
{
    struct usb_mouse *mouse = input_get_drvdata(dev);

    usb_kill_urb(mouse->irq);
}

// Probe function called when a device matching the ID table is found
static int usb_mouse_probe(struct usb_interface *intf, const struct usb_device_id *id)
{
    struct usb_device *dev = interface_to_usbdev(intf);
    struct usb_host_interface *interface;
    struct usb_endpoint_descriptor *endpoint;
    struct usb_mouse *mouse;
    struct input_dev *input_dev;
    int pipe, maxp;
    int error = -ENOMEM;

    interface = intf->cur_altsetting;

    if (interface->desc.bNumEndpoints != 1)
        return -ENODEV;

    endpoint = &interface->endpoint[0].desc;
    if (!usb_endpoint_is_int_in(endpoint))
        return -ENODEV;

    pipe = usb_rcvintpipe(dev, endpoint->bEndpointAddress);
    maxp = usb_maxpacket(dev, pipe);

    mouse = kzalloc(sizeof(struct usb_mouse), GFP_KERNEL);
    input_dev = input_allocate_device();
    if (!mouse || !input_dev)
        goto fail1;

    mouse->data = usb_alloc_coherent(dev, 8, GFP_KERNEL, &mouse->data_dma);
    if (!mouse->data)
        goto fail1;

    mouse->irq = usb_alloc_urb(0, GFP_KERNEL);
    if (!mouse->irq)
        goto fail2;

    mouse->usbdev = dev;
    mouse->dev = input_dev;

    if (dev->manufacturer)
        strscpy(mouse->name, dev->manufacturer, sizeof(mouse->name));

    if (dev->product) {
        if (dev->manufacturer)
            strlcat(mouse->name, " ", sizeof(mouse->name));
        strlcat(mouse->name, dev->product, sizeof(mouse->name));
    }

    if (!strlen(mouse->name))
        snprintf(mouse->name, sizeof(mouse->name),
             "USB HIDBP Mouse %04x:%04x",
             le16_to_cpu(dev->descriptor.idVendor),
             le16_to_cpu(dev->descriptor.idProduct));

    usb_make_path(dev, mouse->phys, sizeof(mouse->phys));
    strlcat(mouse->phys, "/input0", sizeof(mouse->phys));

    input_dev->name = mouse->name;
    input_dev->phys = mouse->phys;
    usb_to_input_id(dev, &input_dev->id);
    input_dev->dev.parent = &intf->dev;

    input_dev->evbit[0] = BIT_MASK(EV_KEY) | BIT_MASK(EV_REL);
    input_dev->keybit[BIT_WORD(BTN_MOUSE)] = BIT_MASK(BTN_LEFT) |
        BIT_MASK(BTN_RIGHT) | BIT_MASK(BTN_MIDDLE);
    input_dev->relbit[0] = BIT_MASK(REL_X) | BIT_MASK(REL_Y);
    input_dev->keybit[BIT_WORD(BTN_MOUSE)] |= BIT_MASK(BTN_SIDE) |
        BIT_MASK(BTN_EXTRA);
    input_dev->relbit[0] |= BIT_MASK(REL_WHEEL);

    input_set_drvdata(input_dev, mouse);

    input_dev->open = usb_mouse_open;
    input_dev->close = usb_mouse_close;

    usb_fill_int_urb(mouse->irq, dev, pipe, mouse->data,
             (maxp > 8 ? 8 : maxp),
             usb_mouse_irq, mouse, endpoint->bInterval);
    mouse->irq->transfer_dma = mouse->data_dma;
    mouse->irq->transfer_flags |= URB_NO_TRANSFER_DMA_MAP;

    error = input_register_device(mouse->dev);
    if (error)
        goto fail3;

    usb_set_intfdata(intf, mouse);
    return 0;

fail3:
    usb_free_urb(mouse->irq);
fail2:
    usb_free_coherent(dev, 8, mouse->data, mouse->data_dma);
fail1:
    input_free_device(input_dev);
    kfree(mouse);
    return error;
}
```

```c
// Disconnect function called when the device is disconnected
static void usb_mouse_disconnect(struct usb_interface *intf)
{
    struct usb_mouse *mouse = usb_get_intfdata (intf);

    usb_set_intfdata(intf, NULL);
    if (mouse) {
        usb_kill_urb(mouse->irq);
        input_unregister_device(mouse->dev);
        usb_free_urb(mouse->irq);
        usb_free_coherent(interface_to_usbdev(intf), 8, mouse->data, mouse->data_dma);
        kfree(mouse);
    }
}

// USB device ID table
static const struct usb_device_id usb_mouse_id_table[] = {
    { USB_INTERFACE_INFO(USB_INTERFACE_CLASS_HID, USB_INTERFACE_SUBCLASS_BOOT,
        USB_INTERFACE_PROTOCOL_MOUSE) },
    { }    /* Terminating entry */
};

MODULE_DEVICE_TABLE (usb, usb_mouse_id_table);  // Macro to export the device ID table

// USB driver structure
static struct usb_driver usb_mouse_driver = {
    .name        = "usbmouse",
    .probe        = usb_mouse_probe,
    .disconnect    = usb_mouse_disconnect,
    .id_table    = usb_mouse_id_table,
};

// Macro to register the USB driver
module_usb_driver(usb_mouse_driver);
```

2. Your USB HID driver wrote with comments

```c
#include <linux/kernel.h>
#include <linux/slab.h>
#include <linux/module.h>
#include <linux/init.h>
#include <linux/usb/input.h>
#include <linux/hid.h>

// Conditional inclusion for Apple device IDs
#ifdef CONFIG_USB_HID_MODULE
#include "../hid-ids.h"
#endif

/*
 * Version Information
 */
#define DRIVER_VERSION "v1.6"
#define DRIVER_AUTHOR "Vojtech Pavlik <vojtech@ucw.cz>"
#define DRIVER_DESC "USB HID Boot Protocol mouse driver"

MODULE_AUTHOR(DRIVER_AUTHOR);        // Sets the author of the module
MODULE_DESCRIPTION(DRIVER_DESC);     // Sets the description of the module
MODULE_LICENSE("GPL");               // Sets the license of the module to GPL

// Structure representing the USB mouse
struct usb_mouse {
    char device_name[128];           // Name of the device
    char device_phys[64];            // Physical path of the device
    struct usb_device *usb_device;   // Pointer to the USB device
    struct input_dev *input_device;  // Pointer to the input device
    struct urb *irq;             // USB request block for interrupt transfer

    signed char *data_buffer;        // Pointer to data buffer
    dma_addr_t data_dma_address;     // DMA address of the data buffer
};

// Interrupt handler for USB mouse
static void usb_mouse_irq(struct urb *urb)
{
    struct usb_mouse *mouse = urb->context;
    signed char *data = mouse->data_buffer;
    struct input_dev *dev = mouse->input_device;
    int status;

    switch (urb->status) {
    case 0:         /* success */
        break;
    case -ECONNRESET: /* unlink */
    case -ENOENT:
    case -ESHUTDOWN:
        return;
    /* -EPIPE:  should clear the halt */
    default:        /* error */
        goto resubmit;
    }
```

```c

    // Report button and movement events
    input_report_key(dev, BTN_LEFT,   data[0] & 0x01);
    input_report_key(dev, BTN_RIGHT,  data[0] & 0x02);
    input_report_key(dev, BTN_MIDDLE, data[0] & 0x04);
    input_report_key(dev, BTN_SIDE,   data[0] & 0x08);
    input_report_key(dev, BTN_EXTRA,  data[0] & 0x10);

    input_report_rel(dev, REL_X,     data[1]);
    input_report_rel(dev, REL_Y,     data[2]);
    input_report_rel(dev, REL_WHEEL, data[3]);

    input_sync(dev);  // Synchronize input events
resubmit:
    status = usb_submit_urb (urb, GFP_ATOMIC);
    if (status)
        dev_err(&mouse->usb_device->dev,
            "can't resubmit intr, %s-%s/input0, status %d\n",
            mouse->usb_device->bus->bus_name,
            mouse->usb_device->devpath, status);
}

// Open function for the input device
static int usb_mouse_open(struct input_dev *dev)
{
    struct usb_mouse *mouse = input_get_drvdata(dev);

    mouse->irq_urb->dev = mouse->usb_device;
    if (usb_submit_urb(mouse->irq_urb, GFP_KERNEL))
        return -EIO;

    return 0;
}

// Close function for the input device
static void usb_mouse_close(struct input_dev *dev)
{
    struct usb_mouse *mouse = input_get_drvdata(dev);

    usb_kill_urb(mouse->irq_urb);
}

// Probe function called when a device matching the ID table is found
static int usb_mouse_probe(struct usb_interface *intf, const struct usb_device_id *id)
{
    struct usb_device *usb_dev = interface_to_usbdev(intf);
    struct usb_host_interface *interface;
    struct usb_endpoint_descriptor *endpoint;
    struct usb_mouse *mouse;
    struct input_dev *input_dev;
    int pipe, maxp;
    int error = -ENOMEM;

    interface = intf->cur_altsetting;

    if (interface->desc.bNumEndpoints != 1)
        return -ENODEV;

    endpoint = &interface->endpoint[0].desc;
    if (!usb_endpoint_is_int_in(endpoint))
        return -ENODEV;

    pipe = usb_rcvintpipe(usb_dev, endpoint->bEndpointAddress);
    maxp = usb_maxpacket(usb_dev, pipe);

    mouse = kzalloc(sizeof(struct usb_mouse), GFP_KERNEL);
    input_dev = input_allocate_device();
    if (!mouse || !input_dev)
        goto fail1;

    mouse->data_buffer = usb_alloc_coherent(usb_dev, 8, GFP_KERNEL, &mouse->data_dma_address);
    if (!mouse->data_buffer)
        goto fail1;

    mouse->irq_urb = usb_alloc_urb(0, GFP_KERNEL);
    if (!mouse->irq_urb)
        goto fail2;

    mouse->usb_device = usb_dev;
    mouse->input_device = input_dev;

    if (usb_dev->manufacturer)
        strscpy(mouse->device_name, usb_dev->manufacturer, sizeof(mouse->device_name));

    if (usb_dev->product) {
        if (usb_dev->manufacturer)
            strlcat(mouse->device_name, " ", sizeof(mouse->device_name)); // Modified here
        strlcat(mouse->device_name, usb_dev->product, sizeof(mouse->device_name)); // Modified here
    }

    if (!strlen(mouse->device_name))
        snprintf(mouse->device_name, sizeof(mouse->device_name),
             "USB HIDBP Mouse %04x:%04x",
             le16_to_cpu(usb_dev->descriptor.idVendor),
             le16_to_cpu(usb_dev->descriptor.idProduct));
```

```c

    usb_make_path(usb_dev, mouse->device_phys, sizeof(mouse->device_phys));
    strlcat(mouse->device_phys, "/input0", sizeof(mouse->device_phys)); // Modified here

    input_dev->name = mouse->device_name;
    input_dev->phys = mouse->device_phys;
    usb_to_input_id(usb_dev, &input_dev->id);
    input_dev->dev.parent = &intf->dev;

    input_dev->evbit[0] = BIT_MASK(EV_KEY) | BIT_MASK(EV_REL);
    input_dev->keybit[BIT_WORD(BTN_MOUSE)] = BIT_MASK(BTN_LEFT) |
        BIT_MASK(BTN_RIGHT) | BIT_MASK(BTN_MIDDLE);
    input_dev->relbit[0] = BIT_MASK(REL_X) | BIT_MASK(REL_Y);
    input_dev->keybit[BIT_WORD(BTN_MOUSE)] |= BIT_MASK(BTN_SIDE) |
        BIT_MASK(BTN_EXTRA);
    input_dev->relbit[0] |= BIT_MASK(REL_WHEEL);

    input_set_drvdata(input_dev, mouse);

    input_dev->open = usb_mouse_open;
    input_dev->close = usb_mouse_close;

    usb_fill_int_urb(mouse->irq_urb, usb_dev, pipe, mouse->data_buffer,
            (maxp > 8 ? 8 : maxp), // Modified here
            usb_mouse_irq, mouse, endpoint->bInterval);
    mouse->irq_urb->transfer_dma = mouse->data_dma_address;
    mouse->irq_urb->transfer_flags |= URB_NO_TRANSFER_DMA_MAP;

    error = input_register_device(mouse->input_device);
    if (error)
        goto fail3;

    usb_set_intfdata(intf, mouse);
    return 0;

fail3:
    usb_free_urb(mouse->irq_urb);
fail2:
    usb_free_coherent(usb_dev, 8, mouse->data_buffer, mouse->data_dma_address);
fail1:
    input_free_device(input_dev);
    kfree(mouse);
    return error;
}

// Disconnect function called when the device is disconnected
static void usb_mouse_disconnect(struct usb_interface *intf)
{
    struct usb_mouse *mouse = usb_get_intfdata(intf);

    usb_set_intfdata(intf, NULL);
    if (mouse) {
        usb_kill_urb(mouse->irq_urb);
        input_unregister_device(mouse->input_device);
        usb_free_urb(mouse->irq_urb);
        usb_free_coherent(interface_to_usbdev(intf), 8, mouse->data_buffer, mouse->data_dma_address); // Modified here
        kfree(mouse);
    }
}

```

Screenshot of USB & mouse information

1. Your second mouse information before unbind original driver (lsusb -tv)

```
user@user-virtual-machine:~$ # echo -n "1-2.2:1.2" > /sys/usb/drivers/usbhid/unbind
user@user-virtual-machine:~$ ksusb -tv
Command 'ksusb' not found, did you mean:
  command 'lsusb' from deb usbutils (1:014-1build1)
Try: sudo apt install <deb name>
user@user-virtual-machine:~$ lsusb -tv
/:  Bus 02.Port 1: Dev 1, Class=root_hub, Driver=ehci-pci/6p, 480M
    ID 1d6b:0002 Linux Foundation 2.0 root hub
/:  Bus 01.Port 1: Dev 1, Class=root_hub, Driver=uhci_hcd/2p, 12M
    ID 1d6b:0001 Linux Foundation 1.1 root hub
    |__ Port 1: Dev 2, If 0, Class=Human Interface Device, Driver=usbhid, 12M
        ID 0e0f:0003 VMware, Inc. Virtual Mouse
    |__ Port 2: Dev 3, If 0, Class=Hub, Driver=hub/7p, 12M
        ID 0e0f:0002 VMware, Inc. Virtual USB Hub
        |__ Port 1: Dev 4, If 1, Class=Wireless, Driver=btusb, 12M
            ID 0e0f:0008 VMware, Inc.
        |__ Port 1: Dev 4, If 0, Class=Wireless, Driver=btusb, 12M
            ID 0e0f:0008 VMware, Inc.
        |__ Port 2: Dev 5, If 3, Class=Human Interface Device, Driver=usbhid, 12M
            ID 0b05:189e ASUSTek Computer, Inc.
        |__ Port 2: Dev 5, If 1, Class=Human Interface Device, Driver=usbhid, 12M
            ID 0b05:189e ASUSTek Computer, Inc.
        |__ Port 2: Dev 5, If 2, Class=Human Interface Device, Driver=usbhid, 12M
            ID 0b05:189e ASUSTek Computer, Inc.
        |__ Port 2: Dev 5, If 0, Class=Human Interface Device, Driver=usbhid, 12M
            ID 0b05:189e ASUSTek Computer, Inc.
```

2. Your second mouse usbid (dmesg)

```
[  276.172485] input: ASUS ROG GLADIUS II WIRELESS as /devices/pci0000:00/0000:00:11.0/0000:02:00.0/usb1/1-2/1-2.2/1-2.2:1.2/0003:0B
05:189E.0004/input/input9
[  276.229538] hid-generic 0003:0B05:189E.0004: input,hidraw3: USB HID v1.11 Keyboard [ASUS ROG GLADIUS II WIRELESS] on usb-0000:02:
00.0-2.2/input2
```

3. Your second mouse information after bind on your own driver (lsusb -tv)

```
user@user-virtual-machine:~$ #echo -n "1-2.2:1.2" > /sys/usb/drivers/usbmouse/bind
user@user-virtual-machine:~$ lsusb -tv
/:  Bus 02.Port 1: Dev 1, Class=root_hub, Driver=ehci-pci/6p, 480M
    ID 1d6b:0002 Linux Foundation 2.0 root hub
/:  Bus 01.Port 1: Dev 1, Class=root_hub, Driver=uhci_hcd/2p, 12M
    ID 1d6b:0001 Linux Foundation 1.1 root hub
    |__ Port 1: Dev 2, If 0, Class=Human Interface Device, Driver=usbhid, 12M
        ID 0e0f:0003 VMware, Inc. Virtual Mouse
    |__ Port 2: Dev 3, If 0, Class=Hub, Driver=hub/7p, 12M
        ID 0e0f:0002 VMware, Inc. Virtual USB Hub
        |__ Port 1: Dev 4, If 1, Class=Wireless, Driver=btusb, 12M
            ID 0e0f:0008 VMware, Inc.
        |__ Port 1: Dev 4, If 0, Class=Wireless, Driver=btusb, 12M
            ID 0e0f:0008 VMware, Inc.
```

Screenshot of left click log messages

1. Left click your mouse 10 times, show your left click printk result (dmesg)

```
[  485.135572] usb 1-2.2: input irq status -75 received
[  485.135580] usb 1-2.2: input irq status -75 received
[  485.135581] usb 1-2.2: input irq status -75 received
[  485.136809] usb 1-2.2: input irq status -75 received
[  485.136814] usb 1-2.2: input irq status -75 received
[  485.136815] usb 1-2.2: input irq status -75 received
[  485.138028] usb 1-2.2: input irq status -75 received
[  485.138031] usb 1-2.2: input irq status -75 received
[  485.138032] usb 1-2.2: input irq status -75 received
[  485.139075] usb 1-2.2: input irq status -75 received
[  485.139079] usb 1-2.2: input irq status -75 received
[  485.139080] usb 1-2.2: input irq status -75 received
```